

INF1022 - Analisadores Léxicos e Sintáticos

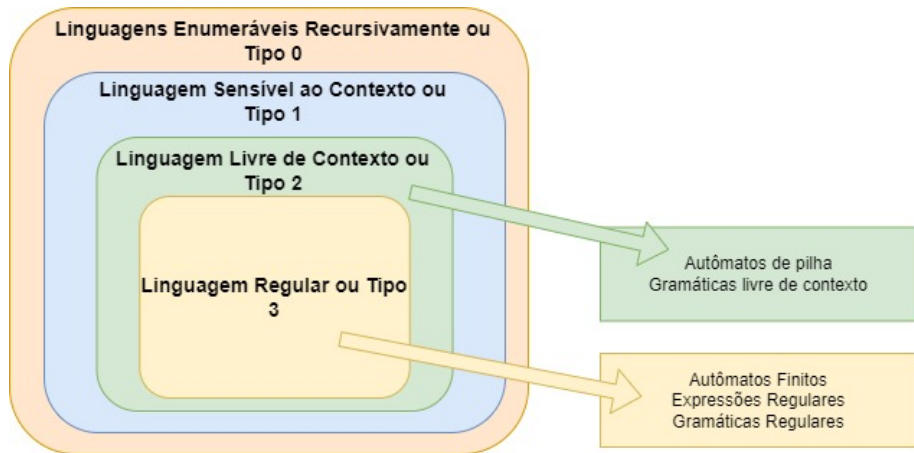
Linguagens Livres de Contexto

Jefferson Santos Edward Hermann Haeusler

PUC-Rio

Departamento de Informática

Hierarquia de Chomsky



Linguagens Livres de Contexto

Regras gramaticais aplicadas em qualquer contexto

Definição

Gramática GLC: $G = \langle \Sigma, \Delta, S, \mathcal{R} \rangle$

- Δ = *Conjunto de símbolos não terminais.*
- Σ = *Conjunto de símbolos terminais.*
- $\mathcal{R}$ = *Conjunto de regras.*
- S = *Símbolo não terminal inicial*

*Uma **Gramática** $\mathcal{G}$ é dita ser Livre de Contexto (GLC), se e somente se, toda regra em $\mathcal{G}$ é da forma $A \rightarrow \gamma$, onde $A \in \Delta_G$ e $\gamma \in (\Sigma_G \cup \Delta_G)^*$*

Linguagens Livres de Contexto

Linguagens Livres de Contexto (LLC)

Definição

Uma **Linguagem** L é dita ser Livre de Contexto, se e somente se, existe uma gramática livre de contexto G , tal que $\mathcal{L}(G) = L$.

Características:

- Aninhamento de estruturas, hierarquia de conceitos, etc.
- Permite o uso de árvores de análise sintática para representar o processo de “parsing”.

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{a^n b^n / n \in \text{Nat}\}$

$$S \rightarrow aSb \mid \epsilon$$

- Gramática para $\{a^k b^n / k > n \in \text{Nat}\}$

$$S \rightarrow aAB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow aBb \mid \epsilon$$

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{a^n b^n / n \in \text{Nat}\}$

$$S \rightarrow aSb \mid \epsilon$$

- Gramática para $\{a^k b^n / k > n \in \text{Nat}\}$

$$S \rightarrow aAB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow aBb \mid \epsilon$$

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{a^n b^n / n \in \text{Nat}\}$

$$S \rightarrow aSb \mid \epsilon$$

- Gramática para $\{a^k b^n / k > n \in \text{Nat}\}$

$$S \rightarrow aAB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow aBb \mid \epsilon$$

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{\gamma/\gamma \text{ não é da forma } \omega\omega, \omega \in \{a, b\}^*\}$

$$S \rightarrow Y|Z|YZ|ZY$$
$$Y \rightarrow a|aYa|aYb|bYa|bYb$$
$$Z \rightarrow b|aZa|aZb|bZa|bZb$$

- Gramática para $\{a^m b^m c^k / m, k \in \text{Nat}\}$

$$S \rightarrow AB$$
$$A \rightarrow aAb|\epsilon$$
$$B \rightarrow cB|\epsilon$$

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{\gamma/\gamma \text{ não é da forma } \omega\omega, \omega \in \{a, b\}^*\}$

$$S \rightarrow Y|Z|YZ|ZY$$

$$Y \rightarrow a|aYa|aYb|bYa|bYb$$

$$Z \rightarrow b|aZa|aZb|bZa|bZb$$

- Gramática para $\{a^m b^m c^k / m, k \in \text{Nat}\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb|\epsilon$$

$$B \rightarrow cB|\epsilon$$

Linguagens Livres de Contexto

Exemplos

- Gramática para $\{\gamma/\gamma \text{ não é da forma } \omega\omega, \omega \in \{a, b\}^*\}$

$$S \rightarrow Y|Z|YZ|ZY$$

$$Y \rightarrow a|aYa|aYb|bYa|bYb$$

$$Z \rightarrow b|aZa|aZb|bZa|bZb$$

- Gramática para $\{a^m b^m c^k / m, k \in \text{Nat}\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb| \epsilon$$

$$B \rightarrow cB| \epsilon$$

Linguagens Livres de Contexto

Exemplos

Gramática para $\{\gamma/\gamma \text{ não é da forma } \omega\omega, \omega \in \{a, b\}^*\}$

$$S \rightarrow Y|Z|YZ|ZY$$

$$Y \rightarrow a|aYa|aYb|bYa|bYb$$

$$Z \rightarrow b|aZa|aZb|bZa|bZb$$

Note que a regra Y gera palavras com comprimento ímpar com centro " a ". E a regra Z gera palavras de comprimento ímpar com centro " b ".

Gramática para $\{a^m b^m c^k / m, k \in \text{Nat}\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb| \epsilon$$

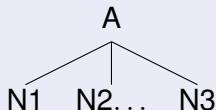
$$B \rightarrow cB| \epsilon$$

Árvores de Análise Sintática

Definição

Dada uma gramática G , uma árvore de análise sintática (AAS) é uma árvore rotulada, onde:

- A raiz é rotulada com o símbolo inicial.
- Folhas são rotuladas com símbolos Terminais.
- Nós internos são rotulados com Não-Terminais.
- Se a árvore contém:



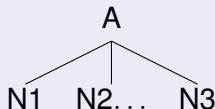
então $A \rightarrow N1N2 \dots N3$ é uma regra da gramática.

Árvores de Análise Sintática

Definição

Dada uma gramática G , uma árvore de análise sintática (AAS) é uma árvore rotulada, onde:

- A raiz é rotulada com o símbolo inicial.
- Folhas são rotuladas com símbolos Terminais.
- Nós internos são rotulados com Não-Terminais.
- Se a árvore contém:



então $A \rightarrow N1 N2 \dots N3$ é uma regra da gramática.

Árvore de uma derivação

A AAS de uma derivação $S \triangleright \gamma_1 \triangleright \gamma_2 \dots \triangleright \gamma_k$ é construída recursivamente a partir de S , a raiz onde cada nível é definido pela respectiva regra.

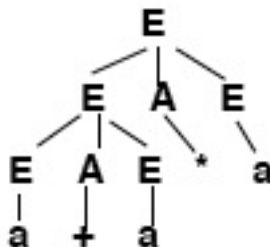
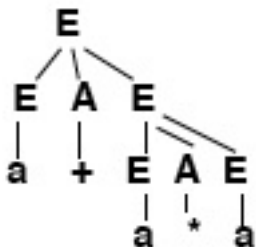
Ambiguidade

Uma gramática G é ambígua, se existe uma palavra $\gamma \in \mathcal{L}(G)$ que possui AASs diferentes.

Ambiguidade

Uma gramática G é ambígua, se existe uma palavra $\gamma \in \mathcal{L}(G)$ que possui AASs diferentes.

A gramática formada pelas regras “ $E \rightarrow a|EAE$ ” e “ $A \rightarrow +|*$ ” é ambígua.

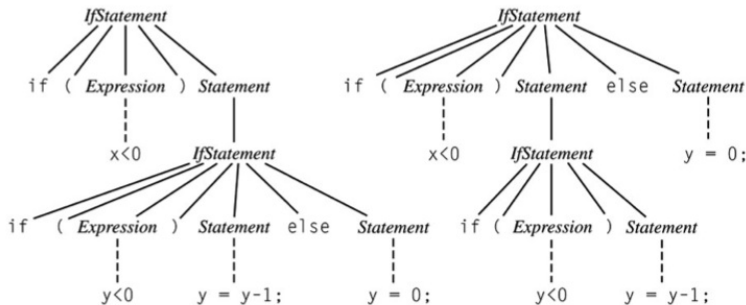


Ambiguidade

Uma gramática G é ambígua, se existe uma palavra $\gamma \in \mathcal{L}(G)$ que possui AASs diferentes.

if-then-else ambíguo

$\text{IfStatement} \rightarrow \text{if (Expression) Statement} \mid$
 $\text{if (Expression) Statement else Statement}$
 $\text{Statement} \rightarrow \text{Assignment} \mid \text{IfStatement}$



Ambiguidade

Uma gramática G é ambígua, se existe uma palavra $\gamma \in \mathcal{L}(G)$ que possui AASs diferentes.

Linguagem Ambígua

Uma LLC é ambígua, se e somente se, toda gramática que a gera é ambígua.

A linguagem $\{a^n b^n c^m d^m / n, m \in \text{Nat}\} \cup \{a^n b^m c^m d^n / n, m \in \text{Nat}\}$ é ambígua. Também chamamos de inerentemente ambígua, quando uma linguagem é ambígua.

Como a união de linguagens livres de contexto é livre de contexto também (**PROVE**), a união acima é uma LLC. Exiba uma gramática livre de contexto que a gere !!!!!

Notações mais econômicas para LLC

- Notação mais próxima de linguagem natural. Usada em Algol 68 (Backus-Naur-Form)
- Os identificadores escritos entre $\langle \text{ e } \rangle$ são não-terminais, os outros símbolos/identificadores são terminais.
- Uso de $|$ para alternativa, de $[]$ para trechos opcionais e $\{\}^*$ para repetições.

$$\begin{aligned}\langle \textit{ident} \rangle &\rightarrow \langle \textit{letter} \rangle \{ \langle \textit{letter} \rangle \mid \langle \textit{digit} \rangle \}^* \\ \langle \textit{procall} \rangle &\rightarrow \langle \textit{ident} \rangle [(\langle \textit{exprlist} \rangle)]\end{aligned}$$

Notações mais econômicas para LLC

- Notação mais próxima de linguagem natural. Usada em Algol 68 (Backus-Naur-Form)
- Os identificadores escritos entre $\langle \text{ e } \rangle$ são não-terminais, os outros símbolos/identificadores são terminais.
- Uso de $|$ para alternativa, de $[]$ para trechos opcionais e $\{\}^*$ para repetições.

$$\begin{aligned}\langle \textit{ident} \rangle &\rightarrow \langle \textit{letter} \rangle \{ \langle \textit{letter} \rangle \mid \langle \textit{digit} \rangle \}^* \\ \langle \textit{procall} \rangle &\rightarrow \langle \textit{ident} \rangle [(\langle \textit{exprlist} \rangle)]\end{aligned}$$

Notações mais econômicas para LLC

- Notação mais próxima de linguagem natural. Usada em Algol 68 (Backus-Naur-Form)
- Os identificadores escritos entre $\langle \text{ e } \rangle$ são não-terminais, os outros símbolos/identificadores são terminais.
- Uso de $|$ para alternativa, de $[]$ para trechos opcionais e $\{ \}^*$ para repetições.

$\langle \textit{ident} \rangle \rightarrow \langle \textit{letter} \rangle \{ \langle \textit{letter} \rangle \mid \langle \textit{digit} \rangle \}^*$
 $\langle \textit{procall} \rangle \rightarrow \langle \textit{ident} \rangle [(\langle \textit{exprlist} \rangle)]$

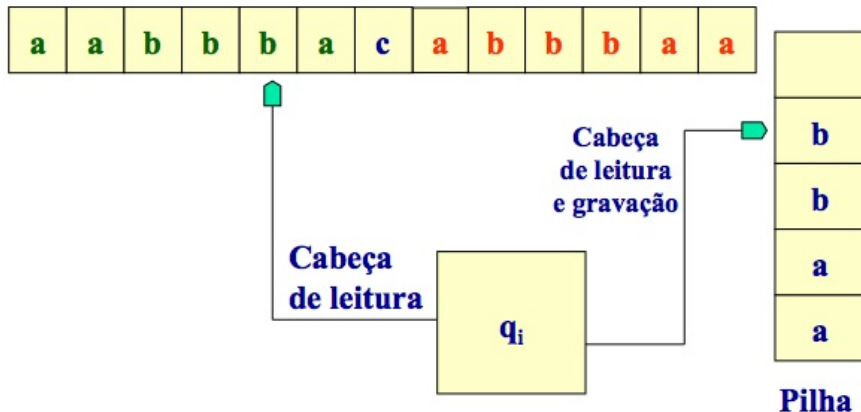
Notações mais econômicas para LLC

- Notação mais próxima de linguagem natural. Usada em Algol 68 (Backus-Naur-Form)
- Os identificadores escritos entre $\langle \text{ e } \rangle$ são não-terminais, os outros símbolos/identificadores são terminais.
- Uso de $|$ para alternativa, de $[]$ para trechos opcionais e $\{ \}^*$ para repetições.

$$\begin{aligned}\langle \textit{ident} \rangle &\rightarrow \langle \textit{letter} \rangle \{ \langle \textit{letter} \rangle \mid \langle \textit{digit} \rangle \}^* \\ \langle \textit{procall} \rangle &\rightarrow \langle \textit{ident} \rangle [(\langle \textit{exprlist} \rangle)]\end{aligned}$$

Aceitadores para LLC

Autômatos com Pilha não-Determinísticos



Aceitadores para LLC

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- Q é um conjunto finito não vazio (de estados).*
- Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- Γ é o alfabeto da pilha (não-vazio e finito).*
- $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- q_0 é o estado inicial e S o símbolo inicial na pilha.*
- F é o conjunto de estados finais.*

Aceitadores para LLC

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Aceitadores para LLC

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Autômatos com Pilha não-Determinísticos (NDPA)

Definição

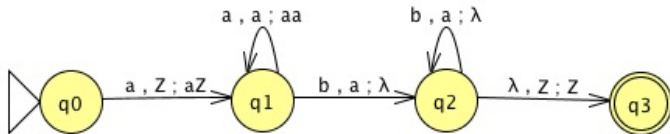
Um autômato de pilha não-determinístico é uma estrutura $\langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$, onde:

- *Q é um conjunto finito não vazio (de estados).*
- *Σ é o alfabeto, conjunto finito não-vazio de símbolos, da linguagem aceita pelo autômato.*
- *Γ é o alfabeto da pilha (não-vazio e finito).*
- *$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \text{pot}(Q \times \Gamma^*)$ é a função de transição.*
- *q_0 é o estado inicial e S o símbolo inicial na pilha.*
- *F é o conjunto de estados finais.*

Aceitadores para LLC

Pilha × Aninhamento

A funcionalidade da pilha dá conta do aninhamento de contexto. O símbolo representando o conceito sintático que é aninhado primeiramente é o último a ser completamente analisado. A notação gráfica σ, N, γ para a transição de q para q_1 , indica o mesmo que $(q_1, \gamma) \in \delta(q, \sigma, N)$. Assim temos o seguinte *NPDA*



para aceitar $\{a^n b^n / n \in \text{Nat}\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{w c w^R / w \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{w / \#a(w) = \#b(w)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{w_1 w_2 / w_1 \neq w_2, \text{ e } w_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{w c w^R / w \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{w / \#a(w) = \#b(w)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{w_1 w_2 / w_1 \neq w_2, \text{ e } w_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Exercício

Fazer (N)PDAs que aceitem as linguagens abaixo

- 1 $\{a^n b^m / m > n \in \text{Nat}\}$
- 2 $\{a^i b^j / i \neq j\}$
- 3 $\{\omega c \omega^R / \omega \in \{a, b\}^*\}$
- 4 $\{a^i b^j c^k / i \neq j\}$
- 5 $\{a^i b^j c^k / j \neq k\}$
- 6 $\{\omega / \#a(\omega) = \#b(\omega)\}$
- 7 $\{a^i b^j c^k / j \neq k \text{ ou } i \neq j\}$
- 8 $\{\omega_1 \omega_2 / \omega_1 \neq \omega_2, \text{ e } \omega_i \in \{a, b\}^*\}$

Critérios de Aceitação por NPDAs

1-Aceitação por estado final

Um NPDA aceita uma palavra ω , se e somente se, está em um estado final ao término da leitura de ω .

Critérios de Aceitação por NPDAs

1-Aceitação por estado final

Um NPDA aceita uma palavra ω , se e somente se, está em um estado final ao término da leitura de ω .

2-Aceitação por pilha vazia

Um NPDA aceita uma palavra ω , se e somente se, está com a pilha vazia (ϵ) ao término da leitura de ω .

Crítérios de Aceitação por NPDAs

1-Aceitação por estado final

Um NPDA aceita uma palavra ω , se e somente se, está em um estado final ao término da leitura de ω .

2-Aceitação por pilha vazia

Um NPDA aceita uma palavra ω , se e somente se, está com a pilha vazia (ϵ) ao término da leitura de ω .

L é aceita pelo critério (1), se e somente se, é aceita pelo critério (2)

(1) $\Rightarrow$ (2)

Se L é aceita por estado final por $\mathcal{A}$ então existe $\mathcal{A}'$ que aceita L por pilha vazia.

Construa $\mathcal{A}'$ como sendo uma cópia de $\mathcal{A}$ com as seguintes transições adicionais, uma para cada símbolo da pilha A , onde q_f é o estado final:

$$\langle q_f, \epsilon, A, q_{pop}, \epsilon \rangle$$

$$\langle q_{pop}, \epsilon, A, q_{pop}, \epsilon \rangle$$

(2) $\Rightarrow$ (1)

Se L é aceita por pilha vazia por $\mathcal{A}$ então existe $\mathcal{A}'$ que aceita L por estado final.

Construa $\mathcal{A}'$ como sendo uma cópia de $\mathcal{A}$ com as seguintes transições adicionais, onde q'_0 é o estado inicial de $\mathcal{A}'$, q_f seu estado final e X_0 um símbolo novo, não existente no alfabeto da pilha de :

$$\langle q'_0, \epsilon, Z, q_0, ZX_0 \rangle$$

$$\langle q, \epsilon, X_0, q_f, X_0 \rangle$$

Z é o símbolo inicial da pilha de $\mathcal{A}$, e temos uma transição do tipo da segunda para cada $q \in Q_{\mathcal{A}}$

Linguagens Determinísticas

Seja $\mathcal{A} = \langle Q, \Sigma, \Gamma, \delta, q_0, S, F \rangle$ um NPDA. Se

- Para cada $q \in Q$ e $\sigma \in \Sigma$ e $N \in \Gamma \cup \{\epsilon\}$, $\text{card}(\delta(q, \sigma, N)) \leq 1$;
- Para cada $q \in Q$ e $N \in \Gamma$, se $\text{card}(\delta(q, \epsilon, N)) = 1$ então para todo $\sigma \in \Sigma$, $\text{card}(\delta(q, \sigma, N)) = 0$.

Então diz-se que $\mathcal{A}$ é determinístico (PDA).

As linguagens dos itens ▶1, ▶2, ▶3, ▶4, ▶5 e ▶6 são determinísticas, pois são aceitas por PDAs.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística ??

SIM, COM CERTEZA !!. Seja $\mathcal{A}$ um PDA que aceita L por estado final. Defina $\bar{\mathcal{A}}$:

- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) . Se $\delta(q, \epsilon, N)$ é definido em $\mathcal{A}$ então não é definido em $\bar{\mathcal{A}}$.
- Acrescente um novo símbolo de pilha I e faça $\delta(q_0, \epsilon, S) = (q_0, SI)$.
- Acrescente, para cada $q \in Q$, as transições $\delta(q, \sigma, I) = (q_f, I)$, onde q_f é um novo estado final.
- Faça $(Q - F) \cup \{q_f\}$ ser o novo conjunto de estados finais.

Observe que o novo PDA aceita $\bar{L}$.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística ??

SIM, COM CERTEZA !!. Seja $\mathcal{A}$ um PDA que aceita L por estado final. Defina $\bar{\mathcal{A}}$:

- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) . Se $\delta(q, \epsilon, N)$ é definido em $\mathcal{A}$ então não é definido em $\bar{\mathcal{A}}$.
- Acrescente um novo símbolo de pilha I e faça $\delta(q_0, \epsilon, S) = (q_0, SI)$.
- Acrescente, para cada $q \in Q$, as transições $\delta(q, \sigma, I) = (q_f, I)$, onde q_f é um novo estado final.
- Faça $(Q - F) \cup \{q_f\}$ ser o novo conjunto de estados finais.

Observe que o novo PDA aceita $\bar{L}$.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística ??

SIM, COM CERTEZA !!. Seja $\mathcal{A}$ um PDA que aceita L por estado final. Defina $\bar{\mathcal{A}}$:

- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) . Se $\delta(q, \epsilon, N)$ é definido em $\mathcal{A}$ então não é definido em $\bar{\mathcal{A}}$.
- Acrescente um novo símbolo de pilha I e faça $\delta(q_0, \epsilon, S) = (q_0, SI)$.
- Acrescente, para cada $q \in Q$, as transições $\delta(q, \sigma, I) = (q_f, I)$, onde q_f é um novo estado final.
- Faça $(Q - F) \cup \{q_f\}$ ser o novo conjunto de estados finais.

Observe que o novo PDA aceita $\bar{L}$.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística ??

SIM, COM CERTEZA !!. Seja $\mathcal{A}$ um PDA que aceita L por estado final. Defina $\bar{\mathcal{A}}$:

- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) . Se $\delta(q, \epsilon, N)$ é definido em $\mathcal{A}$ então não é definido em $\bar{\mathcal{A}}$.
- Acrescente um novo símbolo de pilha I e faça $\delta(q_0, \epsilon, S) = (q_0, SI)$.
- Acrescente, para cada $q \in Q$, as transições $\delta(q, \sigma, I) = (q_f, I)$, onde q_f é um novo estado final.
- Faça $(Q - F) \cup \{q_f\}$ ser o novo conjunto de estados finais.

Observe que o novo PDA aceita $\bar{L}$.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística ??

SIM, COM CERTEZA !!. Seja $\mathcal{A}$ um PDA que aceita L por estado final. Defina $\bar{\mathcal{A}}$:

- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) . Se $\delta(q, \epsilon, N)$ é definido em $\mathcal{A}$ então não é definido em $\bar{\mathcal{A}}$.
- Acrescente um novo símbolo de pilha I e faça $\delta(q_0, \epsilon, S) = (q_0, SI)$.
- Acrescente, para cada $q \in Q$, as transições $\delta(q, \sigma, I) = (q_f, I)$, onde q_f é um novo estado final.
- Faça $(Q - F) \cup \{q_f\}$ ser o novo conjunto de estados finais.

Observe que o novo PDA aceita $\bar{L}$.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística!

Em outras palavras, para construir o PDA complementar ($\bar{\mathcal{A}}$) que consegue aceitar a linguagem complementar a aceita pelo $\mathcal{A}$, faça:

- A aceitação do autômato $\bar{\mathcal{A}}$ será por estado final. Faça a aceitação do automato $\mathcal{A}$ também ser por estado final.
- Crie um novo estado inicial q_0 que incluía um novo símbolo na pilha (I) que fique abaixo do símbolo inicial anterior (Z).
 $\delta(q_0, \epsilon, S) = (q_0, SI)$
- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) em todos os estados. Caso você adicione triplas, toda tripla adicionada a $\mathcal{A}$, vai levar ao estado LIXO. O estado LIXO deve ser final e deve conter um loop para ele mesmo com todas as triplas possíveis.
- Faça todos os estados finais de $\mathcal{A}$ serem não finais em $\bar{\mathcal{A}}$ e todos os estados não finais serem finais.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística!

Em outras palavras, para construir o PDA complementar ($\bar{\mathcal{A}}$) que consegue aceitar a linguagem complementar a aceita pelo $\mathcal{A}$, faça:

- A aceitação do autômato $\bar{\mathcal{A}}$ será por estado final. Faça a aceitação do automato $\mathcal{A}$ também ser por estado final.
- Crie um novo estado inicial q_0 que incluía um novo símbolo na pilha (I) que fique abaixo do símbolo inicial anterior (Z).
 $\delta(q_0, \epsilon, S) = (q_0, SI)$
- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) em todos os estados. Caso você adicione triplas, toda tripla adicionada a $\mathcal{A}$, vai levar ao estado LIXO. O estado LIXO deve ser final e deve conter um loop para ele mesmo com todas as triplas possíveis.
- Faça todos os estados finais de $\mathcal{A}$ serem não finais em $\bar{\mathcal{A}}$ e todos os estados não finais serem finais.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística!

Em outras palavras, para construir o PDA complementar ($\bar{\mathcal{A}}$) que consegue aceitar a linguagem complementar a aceita pelo $\mathcal{A}$, faça:

- A aceitação do autômato $\bar{\mathcal{A}}$ será por estado final. Faça a aceitação do automato $\mathcal{A}$ também ser por estado final.
- Crie um novo estado inicial q_0 que incluía um novo simbolo na pilha (I) que fique abaixo do simbolo inicial anterior (Z).
 $\delta(q_0, \epsilon, S) = (q_0, SI)$
- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) em todos os estados. Caso você adicione triplas, toda tripla adicionada a $\mathcal{A}$, vai levar ao estado LIXO. O estado LIXO deve ser final e deve conter um loop para ele mesmo com todas as triplas possíveis.
- Faça todos os estados finais de $\mathcal{A}$ serem não finais em $\bar{\mathcal{A}}$ e todos os estados não finais serem finais.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística!

Em outras palavras, para construir o PDA complementar ($\bar{\mathcal{A}}$) que consegue aceitar a linguagem complementar a aceita pelo $\mathcal{A}$, faça:

- A aceitação do autômato $\bar{\mathcal{A}}$ será por estado final. Faça a aceitação do automato $\mathcal{A}$ também ser por estado final.
- Crie um novo estado inicial q_0 que incluía um novo simbolo na pilha (I) que fique abaixo do simbolo inicial anterior (Z).
 $\delta(q_0, \epsilon, S) = (q_0, SI)$
- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) em todos os estados. Caso você adicione triplas, toda tripla adicionada a $\mathcal{A}$, vai levar ao estado LIXO. O estado LIXO deve ser final e deve conter um loop para ele mesmo com todas as triplas possíveis.
- Faça todos os estados finais de $\mathcal{A}$ serem não finais em $\bar{\mathcal{A}}$ e todos os estados não finais serem finais.

Linguagens Determinísticas

Se L é determinística então $\bar{L}$ é determinística!

Em outras palavras, para construir o PDA complementar ($\bar{\mathcal{A}}$) que consegue aceitar a linguagem complementar a aceita pelo $\mathcal{A}$, faça:

- A aceitação do autômato $\bar{\mathcal{A}}$ será por estado final. Faça a aceitação do automato $\mathcal{A}$ também ser por estado final.
- Crie um novo estado inicial q_0 que incluía um novo símbolo na pilha (I) que fique abaixo do símbolo inicial anterior (Z).
$$\delta(q_0, \epsilon, S) = (q_0, SI)$$
- Complete $\mathcal{A}$ para que seja definido para toda tripla (q, σ, N) em todos os estados. Caso você adicione triplas, toda tripla adicionada a $\mathcal{A}$, vai levar ao estado LIXO. O estado LIXO deve ser final e deve conter um loop para ele mesmo com todas as triplas possíveis.
- Faça todos os estados finais de $\mathcal{A}$ serem não finais em $\bar{\mathcal{A}}$ e todos os estados não finais serem finais.

Linguagens Determinísticas

Toda Linguagem Livre de Contexto é aceita por PDA ??

NÃO. Veremos que o complemento das linguagens dos itens ▶ 7 e ▶ 8 não são LLC

Linguagens Determinísticas

Toda Linguagem Livre de Contexto é aceita por PDA ??

NÃO. Veremos que o complemento das linguagens dos itens  e  não são LLC

Toda Linguagem Livre de Contexto é aceita por NPDA ??

Linguagens Determinísticas

Toda Linguagem Livre de Contexto é aceita por PDA ??

NÃO. Veremos que o complemento das linguagens dos itens  e  não são LLC

Toda Linguagem Livre de Contexto é aceita por NPDA ??

SIM

Equivalência de LLC e NPDAs (I)

Teorema

Seja $\mathcal{G} = \langle \Sigma, \Gamma, S, \mathcal{R} \rangle$ uma Gramática Livre de Contexto. Existe um NPDA $\mathcal{A}$, tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{G})$.

Prova Construa $\mathcal{A} = \langle \{q_0\}, \Sigma, \Gamma, \delta, q_0, S, \{q_0\} \rangle$ e δ é definido como:

$$\begin{array}{lll} (q_0, \gamma) \in \delta(q_0, \sigma, N) & \text{se e somente se} & N \rightarrow \sigma\gamma \in \mathcal{R} \\ (q_0, V\gamma) \in \delta(q_0, \epsilon, N) & \text{se e somente se} & N \rightarrow V\gamma \in \mathcal{R}, V \in \Gamma \\ \delta(q_0, \sigma, \sigma) = (q_0, \epsilon) & \text{se e somente se} & \sigma \in \Sigma \end{array}$$

Exemplo de um NPDA com um só estado escrito direto de uma GLC

Exemplo: $Z \rightarrow Z + Z | Z * Z | (Z) | a | b$

$*$, $*$; λ
 $+$, $+$; λ
) ,) ; λ
(, (; λ
 b , b ; λ
 a , a ; λ
 λ , Z ; b
 λ , Z ; a
 λ , Z ; (Z)
 λ , Z ; $Z * Z$
 λ , Z ; $Z + Z$

